BS - KW3

- Scheduling
- Despatcher, Scheduler

Der Dispatcher ist ein Programm, welches die Umschaltung zwischen Prozessen realisiert. Der Scheduler ist ein Programm, welches bestimmt, zu welchem Prozess gewechselt wird.

↳ Prozess Scheduling: Planung der zeitlichen Zuteilung von Prozessen
↳ Umschaltungsvorgang heißt Prozess- / Kontextwechsel

⇒ Ziele:

Anwender:
- Kurze Antwortzeiten bei interaktiven Aufträgen
- Lange Verweilzeiten für Stapelverarbeitung

BS:
- Hoher Durchsatz
- Hohe Auslastung der Ressourcen
- Fairness in der Behandlung aller Aufträge
- Scheduling soll gering Aufwand verursachen

- Präemptiv und Nicht-Präemptives Scheduling

Beim präemptiven Scheduling kann ein Prozess jederzeit die CPU entzogen werden.
Beim nicht-präemptiven Scheduling hat ein Prozess solange die CPU, bis es endet oder freiwillig die CPU abgibt

↳ Präemptives Scheduling verursacht größeren Aufwand, erlaubt aber Bearbeitung einer höheren Anzahl von Prozessen

↳ Verfahren, bei denen ein Prozess auf eigenen Wunsch die CPU abgibt, nennt man Kooperatives Scheduling.

BS - KVB

→ Spezielle Scheduling Verfahren

- Prioritätenbasiertes Scheduling:
  Jeder Prozess bekommt Prioritätsstufe gemäß seiner relativen Wichtigkeit zugeordnet.
  ↳ Präemptiv oder nicht
- Zeitscheibenbasiertes Scheduling: Prozessorzeitteilung auf Basis gewählter Zeitspannen mittels Uhrunterbrechungen
  ↳ präemptive

→ Nicht-Präemptive Verfahren:

- First-Come-First-Served (FCFS)
  ↳ Vollständige Bearbeitung jedes Prozesses, bevor ein neuer beginnt
  ↳ außer Prozess blockiert → Prozesswechsel
  ↳ Kritik: → Benachteiligung kurzer Prozesse gegenüber längeren Prozessen
  → Begünstigung CPU-lastiger Prozesse gegenüber E/A-lastigen
  → Für Einprozessorsystem nicht geeignet
- Shortest-Process-Next (SPN)
  ↳ von allen rechenwilligen Prozessen wird der mit der kürzesten Bearbeitungszeitanforderung ausgewählt
  ↳ Bei gleicher Zeit ⇒ FCFS
  ↳ SPN garantiert kürzeste mittlere Wartezeit
  ↳ Lange Prozesse können Verhungern.

Betriebssysteme

Spezielle Scheduling-Verfahren

→ Prioritätsbasiertes Scheduling

Jeder Prozess bekommt Prioritätsstufe seiner relativen Wichtigkeit zugeordnet

→ Prioritäten können

→ extern oder intern durch BS bestimmt werden

→ statisch oder dynamisch sein

→ Präemptives oder nicht präemptives

→ Zeitscheibenbasiertes Scheduling: Prozesszuteilung auf Basis gewählter Zeitspannen mittels Uhrunterbrechungen

→ Präemptives Verfahren

⇒ Kürzeste Zeit: Prozesswechsel bei Ablauf einer Zeitscheibe oder nach Zustandswechsel eines höherprioren Prozesses von blockiert in rechenwillig

Scheduling

Nicht-präemptive Verfahren

→ Anwendung: Stapeljobs mit erfahrenen Bedienenden

→ Verfahren: First-Come-First-Served (FCFS)
Shortest-Process-Next (SPN)
Prioritäts-Scheduling (Prio)

Beispiel Folie 383

First Come First Served

→ Vollständige Bearbeitung jedes Prozesses, bevor ein neuer begonnen wird: "FIFO"

→ Ausnahme: Prozess ist blockiert, dann Prozesswechsel

→ Implementierung: Ready-Queue wird als FIFO-Liste verwaltet!

1. Beispiel:
→ P A ist rechenintensiv, nächste E/A Operation in 30s
→ P B ist C und E/A-lastig, Rechenphasen von 30 ms
→ Reihenfolge der 8 Prozesse hat starken Einfluss auf Wartezeit

Kritik: → Benachteiligung kurzer Prozesse gegenüber langen Prozessen
→ Begünstigung CPU-lastiger Prozesse gegenüber E/A-lastigen Prozessen
→ Für Einprozessorsystem nicht geeignet
→ FCFS wird in Kombination mit Prioritäten genutzt.

Shortest-Process-Next

→ Von allen rechenwilligen Prozessen wird der mit der kürzesten Bearbeitungszeitanforderung ausgewählt.

→ Bei gleichen Bearbeitungsanforderungen: FCFS

→ SPN bestimmt garantiert die kürzeste mittlere Wartezeit für alle Aufträge

→ Bedingung: Bearbeitungszeit muss bekannt sein

→ Bevorzugung langer Prozesse wird vermieden

→ Gefahr: lange Prozesse verhungern!

Prioritäts-Scheduling (Pro-)

→ Jeder Auftrag bekommt feste Priorität zugeordnet

→ Von allen rechenwilligen Prozessen wird der mit der höchsten Priorität ausgewählt

→ Gleiche Priorität: FCFS

→ Problem: Prozesse mit geringer Priorität verhungern, wenn stets Prozess mit höherer Priorität ablaufbereit ist.

→ Alternative: Erhöhung der Priorität mit zunehmendem „Alter"

Betriebssysteme

Präemptive Scheduling-Verfahren

Round-Robin-Scheduling (RR)

- → Zeitscheibenbasiertes Verfahren
- → Rechenwillige Prozesse werden in Reihenfolge geordnet
- → Jeder rechenwillige Prozess erhält Prozessor für eine feste Zeitdauer q (Zeitscheibe, time slice, Quantum)
- → Nach Ablauf der Zeitscheibe erfolgt Prozesswechsel
- → Tritt vor Ende der Zeitscheibe eine Blockierung oder Prozessende ein, so erfolgt der Prozesswechsel sofort.

- → Implementierung:
  - → Zeitscheibe wird durch Uhr-Interrupt realisiert
  - → Ready Queue wird als lineare Liste verwaltet

→ Sind n Prozesse in der Ready Queue, so bekommt jeder Prozess $\frac{1}{n}$ der CPU-Zeit in Einheiten der Größe q

→ Kein Prozess wartet länger als (n-1) q Zeiteinheiten

→ Fair!

→ Unterschied: Wahl von q

- → q zu klein: häufiger Prozesswechsel, Prozessumschaltung nützt
- → q zu groß: es ergeben sich schlechte Antwortzeiten.

Betriebssysteme

Unterbrechendes Prioritäts-Scheduling

-> Jeder Prozess erhält beim Start eine Priorität
-> Prozesse werden gemäß ihrer Priorität in Queue eingereiht.
-> Es wird jeweils der rechenwillige Prozess mit höchster Priorität ausgewählt und bedient.
-> Wird Prozess höherer Priorität rechenwillig so wird laufender Prozess unterbrochen und in Queue eingefügt.

Quiz

Länge q bei RR?

-> Die Auslastung der CPU durch Anwendungscode steigt mit wachsendem q.

BS - HÜB

→ Nicht-Präemptive Verfahren

- Prioritäts-Scheduling (Prio)
  - → Jeder Auftrag bekommt feste Priorität zugeordnet
  - → Von allen rechenwilligen Prozessen wird der mit der höchsten Priorität ausgewählt
  - → Gleiche Prio → FCFS
  - → Problem: geringere Prio verhungern

→ Präemptive Verfahren

- Round-Robin-Scheduling (RR)
  - → zeitscheibenbasiertes Verfahren
  - → Rechenwillige Prozesse werden in fester Reihenfolge geordnet
  - → Jeder rechenwillige Prozess erhält Prozessor für eine feste Zeitdauer q
  - → Nach Ablauf der Zeitscheibe erfolgt Prozesswechsel
  - → Tritt vor Ende der Zeitscheibe eine Blockierung oder Prozessende ein, so erfolgt der Prozesswechsel sofort.
  - → Implementierung: → Zeitscheibe wird durch Uhr-Interrupt realisiert
    → Ready-Queue wird als lineare Liste verwaltet
  - → Vorgehen
    - → Prozess wird aus Queue entfernt und bekommt Prozessor zugeordnet
    - → Am Ende der Zeitscheibe oder wenn Prozess blockiert erfolgt Prozesswechsel, Prozess wird am Ende der Queue eingefügt
    - → Während oder bei Ende der Zeitscheibe eintreffende Aufträge werden am Ende der Queue eingefügt

BS - VUB

~> Round-Robin

-> Sind n Prozesse in der Ready Queue, so bekommt jeder Prozess 1/n der CPU-Zeit in Einheiten der Größe q

-> Kein Prozess wartet länger als (n-1) q Zeiteinheiten

-> RR ist einfach und weit verbreitet

-> Fair

-> Kritischer Punkt: -> q zu klein: häufige Prozesswechsel, Prozessornutzung sinkt

-> q zu groß: schlechte Antwortzeiten

~> Unterbrechendes Prioritäts Scheduling

-> Jeder Prozess erhält beim Start eine Priorität

-> Prozesse werden gemäß ihrer Priorität in Queue eingereiht

-> Jeweils Prozess mit Höchster Priorität ausgeführt

BS - NUB

~> Dateisysteme

- Flüchtiger vs Persistenter Speicher

Ein flüchtiger Speicher erfordert eine permanente Energiezufuhr, sonst gehen Daten verloren.
Ein persistenter Speicher speichert Daten dauerhaft ohne Energiezufuhr.

flüchtig: RAM , persistent: Flash-Speicher

~> Solid-State-Disk (SSD)

-> Nicht flüchtiges elektronisches Speichermedium
-> Keine mechanischen Bauteile: -> mechanisch robuster
-> kurze Zugriffszeiten
-> keine Geräusche
-> Blöcke können nur als Ganzes beschrieben werden
-> Wear-Leveling: Controller verteilt Schreibvorgänge auf alle Speicherzellen

~> Schnittstellen

-> SATA für HDD
-> NVMe für SSD

BS - IV/B

- **Datei (File)**
  Logische Einheit zur Speicherung von Informationen auf externen Speichermedien mit Namen und Verwaltungsinformation

→ Dateiverwaltungssystem:
- stellt Anwendern Dienste zur Nutzung von Dateien zur Verfügung
- vereinheitlicht persistenten Speicher gegenüber Anwendungen

→ Ziel: Für Benutzer ist es transparent, wie Speicher im Detail funktioniert und wo die Daten abgelegt werden.

→ BS stellt Anwendungen logische Sicht zur Verfügung und realisiert diese effizient

⇝ Aufgaben eines Dateiverwaltungssystems
- → API für Umgang mit Dateien anbieten
- → Integrität: verhindern, dass Dateien verloren gehen oder beschädigt werden
- → E/A mit einer Vielzahl von Speichergeräten unterstützen
- → Gleichzeitige E/A-Aufträge mehrerer Prozesse koordinieren
- → Leistungsverhalten optimieren

⇝ Dateistrukturen:
- Unstrukturierte Dateien: Folge von Bytes
  ↳ Interpretation des Inhalts kann nur durch Anwendung erfolgen
- Strukturierte Dateien: Folge von Datensätzen fester Länge
  ↳ Relationale Datenbanken haben den Einsatz von strukturierten Dateien verdrängt

BS - VL VB

→ Softwarearchitektur des Dateisystems

- Gerätetreiber: Kommunikation mit Peripheriegeräten, Controllern, startet E/A-Operationen
- Einfaches Dateisystem
  - ↳ Verwaltung von Datenblöcken, in die persistenter Speicher unterteilt ist
  - ↳ Transfer von Blöcken zwischen Hauptspeicher & persistentem Speicher
  - ↳ Dateiinhalt oder -struktur sind hier nicht bekannt
- E/A-Supervisor
  - ↳ Zuordnung von Dateien zu Geräten
  - ↳ Scheduling von Zugriffen auf externe Geräte
  - ↳ E/A-Puffer
- Logische E/A
  - ↳ Zugriff auf logische Einheiten
- Zugriffsverfahren
  - ↳ API für Anwendungen

→ Dateiorganisation & Dateizugriff

Kriterien:
- → Schneller Zugriff
- → Einfache Aktualisierung
- → Ökonomische Speicherung
- → Einfache Verwaltung
- → Zuverlässigkeit

BS – KW2

~> Zugriffsarten

- Sequentieller Zugriff
  - -> Verarbeitung vom Dateianfang fortlaufend bis Dateiende
  - -> Zurückspringen auf Dateianfang
- Direkter Zugriff:
  - -> Bytes oder Datensätze der Datei können in beliebiger Reihenfolge verarbeitet, gelesen/geschrieben werden

~> asynchroner Dateizugriff

Lese- & Schreiboperationen nicht-blockierend

~> Memory Mapped File

-> In virtuellen Adressraum eines Prozesses abgebildete Datei

-> Daten lesen/schreiben ohne Funktionen read() / write()

-> Vorteile:

(+) Schnellere Dateizugriffe, da kein Zugriff auf externe Medien notwendig und keine Systemaufrufe

(+) Einfacher Dateizugriff mit Hilfe von Zeigern

(+) Mehrere Prozesse können die Datei durch direktes Lesen und Schreiben im Hauptspeicher ändern

-> Nachteile:

(-) treffende Seitenfehler können zur Verzögerung führen

BS - KVD

→ Metadaten: Jede Datei wird durch Metadaten (Dateiattribute) beschrieben. (zB, Rechte, Typ, Größe, etc.)

- Verzeichnis (Ordner, Directory)
  Ein Verzeichnis dient der Strukturierung aller Dateien eines Dateisystems in einer Hierarchie. Es enthält Einträge für Dateien und Verzeichnisse.

→ Mountbare Verzeichnisbäume
- → Der für Anwender sichtbare Dateiraum ist oft durch mehrere Dateisysteme auf mehreren Speichern/Rechnern realisiert.
- → Alternativen:
  - Dateinamen spiegeln die verschiedenen Speicher wieder
    eg: "C:\..." or "D:\..."
  - Transparenz im Dateinamensraum
    - → Gesamt-Dateiraum wird aus mehreren Dateisystemen mit jeweils eigenem Verzeichnisbaum durch Mountern gebildet
    - → Nach dem Mountern existiert ein einziger Dateinamensbaum (mounten!)

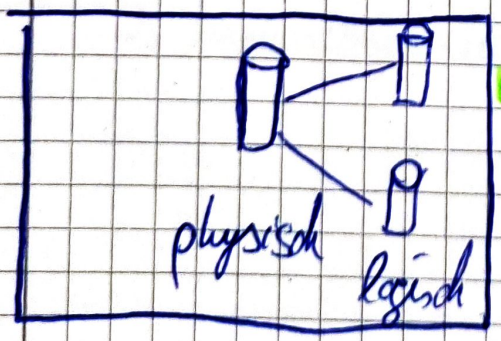
↪ Logical Volume Manager (LVM)

- LVM: Abstrahiert und dynamische Verwaltung von Datenspeichern
  - ↳ Logical Volumes: Bereiche, die sich über mehrere physische Laufwerke erstrecken
- Vorteile: (+) Volumes können im laufenden Betrieb vergrößert, angelegt und verkleinert werden
  (+) Kapazität mehrerer Festplatten kann zu einem Volume zusammengefasst werden.
  (+) LVMs haben hohen Datendurchsatz und beeinträchtigen System nur unwesentlich
- Nachteil: (-) Liegt Logical Volume auf mehreren Festplatten, so sind beim Ausfall einer Platte alle Daten verloren

● Partitionen

Eine Partition ist ein logischer Teil einer Festplatte, der wie eine physikalische Festplatte angesprochen wird

↳ Partitionierungsinformationen wird auf physischem Laufwerk gespeichert
↳ Jede Partition kann genau ein Dateisystem aufnehmen



Vorteile: (+) Jede Partition lässt sich einzeln neu formatieren, wobei die Inhalte der anderen Partitionen erhalten bleiben

(+) HW-Fehler einer Partition lassen andere Partitionen unbeeinflusst
(+) In jeder Partition kann eigenes BS installiert werden
(+) Strukturierung nach BS/Benutzerdaten

BS – VL 16

⇝ Organisation in Blöcken

- → Festplatten sind in Partitionen und Blöcke unterteilt
- → Blöcke sind Zugriffseinheiten bei E/A auf persistenten Speicher
- → Blöcke werden auf die spezifischen Eigenschaften des Speichermediums abgebildet
- → Große Blöcke gut für sequentielle Verarbeitung, bei wahlfreiem Zugriff verlieren große Blöcke das Lokalitätsprinzip

- **Fragmentierung von Festplatten**

  Fragmentierung bezeichnet den Effekt, dass zusammenhängende Dateien nicht am Stück auf der Festplatte gespeichert sind, sondern verstreut über die ganze Festplatte

⇝ Verwaltung des persistenten Speichers

Aufgaben

- → Verwaltung der Plattenblöcke und Zugehörigkeit zu einer Datei
- → Dateien benannten Platzt auf persistentem Speicher zugeordnet
- → Verwaltung des belegten Platz und des freien Speichers

BS - UB

Kontinuierliche Allokation

- Datei wird eine Folge zusammenhängender Plattenblöcke zugeordnet
- Vorteile: (+) Einfache Implementierung
  (+) Sehr gute Performance beim Lesen / Schreiben
- Nachteile: (-) Maximale Größe muss beim Anlegen der Datei festgelegt werden
  (-) Fragmentierung
  (-) Verdichtung / Defragmentierung sehr aufwändig

Allokation mit verketteter Liste

- Datei wird verkettete Liste von Plattenblöcken zugeordnet
- Erstes Wort jedes Blockes enthält Verweis auf nächsten Block, der Rest enthält Daten
- Vorteile: (+) Einfache Implementierung
  (+) Keine externe Fragmentierung
  (+) Datei kann vergrößert und verkleinert werden
- Nachteile: (+) Wahlfreier Zugriff ist sehr langsam
  (-) Zuverlässigkeitsprinzip verletzt, periodische Reorganisation
  (-) Fehleranfälligkeit

BS - HVB

→ Allokation mit verketteter Liste und Index

→ Datei wird verkettete Liste von Plattenblöcken zugeteilt

→ Speicherung der Verkettung in separater Tabelle (Index)

→ Vorteile: (+) Gesamter Datenblock steht für Daten zur Verfügung

(+) Gute Performance bei direktem Zugriff

(+) Für Verzeichniseintrag wird nur Adresse des Blockes mit dem Index gespeichert

→ Nachteile: (-) Gesamter Index aller Dateien muss gehalten werden

→ Allokation mit Index-Knoten

→ Index-Knoten (i-node): Dateikontrollblock, der neben den Dateiattributen eine kleine Tabelle mit Adressen von zugeordneten Plattenblöcken enthält

→ Kapazität Blocknummern von Datei:

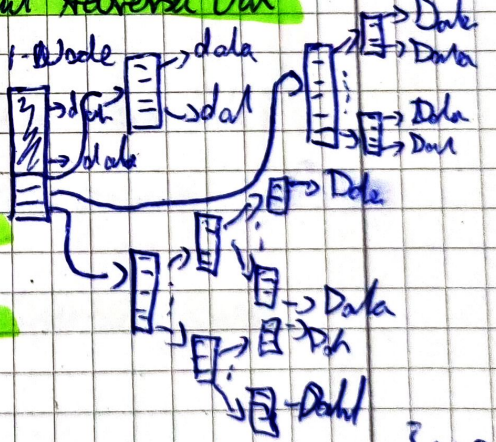
Fortlaufend begrenzt bei direktadressierten Blöcken

→ Wenige Adressen direkt in I-Node gespeichert

→ Beim Öffnen von Datei wird I-Node in Hauptspeicher geladen, dann sind die ersten Blockadressen direkt verfügbar

→ Kleine Dateien: schneller Zugriff

→ sinkende Zugriffsgeschwindigkeit bei wachsender Dateigröße.



$(12 + 256 + 256^2 + 256^3) * \text{Blockgröße}$

BS - WH

- **Sparse Files**

Ein Sparse File bezeichnet eine Datei, die Bereiche mit unbestimmten Inhalt oder Nullbytes enthält.

→ Sparse files können sehr kompakt gespeichert werden

→ Es werden nur die Blöcke im Dateisystem angelegt, die auch tatsächlich Daten enthalten

→ Nicht benötigte Verwaltungsstrukturen werden weggelassen

- **Journaling - Dateisystem**

Bei Journaling - Dateisystemen kann zu jedem Zeitpunkt ein konsistenter Dateizustand rekonstruiert werden, auch wenn Schreibvorgang an beliebiger Stelle abgebrochen wurde

→ Änderungen werden vor eigentlichem Schreiben in reservierten Speicherbereich, dem Journal, aufgezeichnet

→ Vorteile: (+) Auch beim Abbruch eines Schreibvorgangs ist Dateisystem konsistent

→ Nachteil: (-) Schreiboperationen dauern länger

BS - RAID

⇝ RAID Systeme

- Redundant Array of Independent Disks (RAID)
  Mehrere Festplatten sind zu einem logischen Verbund zusammengefasst. Dieser bietet die gleiche API wie eine einzige Festplatte an, d.h. Aufteilung in Festplatten ist nach außen vollständig transparent.

↳ Daten werden auf mehreren Platten verteilt und redundant gespeichert

↳ Erhöhung der
  → Zugriffsgeschwindigkeit durch parallales Lesen und Schreiben
  → Zuverlässigkeit durch Mehrfachspeicherung der Daten auf verschiedenen Platten.

↳ Ablauf:
  → Bei logischer E/A Anfrage an RAID, wird bestimmt von welchen physischen Festplatten gelesen werden muss und physikalische E/A Anfragen werden erzeugt.
  → Aus Antworten der physikalischen E/A Anfragen wird Antwort für logische E/A Anfragen gebildet

↳ Hardware RAID Controller
  → HW Baustein setzt RAID Funktionalität um
  → Keine Zusatzbelastung für CPU des Rechners durch RAID-Berechnungen
  → Ermöglicht das direkte Booten von BS von einem RAID-System

↳ Software RAID Controller
  → Software zur Steuerung der Platten läuft auf CPU des Host Rechners
  → Kostengünstig aber hohe CPU Last

BS - VUB

→ RAID 0 (Striping)

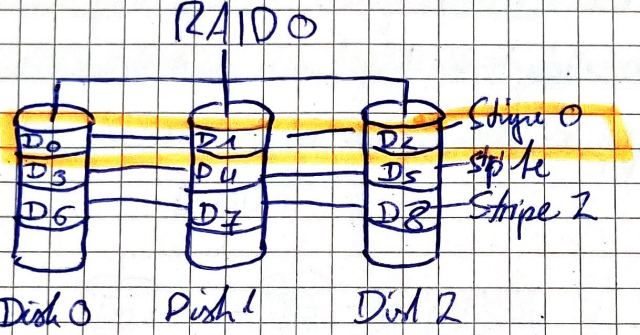
→ Beschleunigung des Zugriffs auf Daten

→ Festplatten-Verbund von zwei oder mehr Festplatten

→ Beteiligte Festplatten sind in zusammenhängende Blöcke (Stripes) fester Größe aufgeteilt

→ Daten werden in Blöcke gleicher Größe aufgeteilt und ein Reißverschlussverfahren (Striping) auf Blöcke der Festplatten aufgeteilt.

→ Zugriff auf alle Festplatten können parallel durchgeführt werden



→ Vorteile: (+) Schreib/Lesevorgänge auf mehreren Festplatten verteilt

(+) Beschleunigungsfaktor ≈ Anzahl der Platten

→ Effektive Zugriffsrate hängt von Zugriffsart und Blockgröße ab

→ Nachteil: (-) Fällt eine Festplatte aus, dann sind praktisch alle Daten weg

(-) Zuverlässigkeit sinkt mit Anzahl der Platten

Einsatz: - Anwendungsbereich zur Geschwindigkeitssteigerung bei sequentiellen Datentransfers

- wenn Ausfallsicherheit keine Bedeutung hat

- zB Audio / Video

- Temporäre Zwischenspeicherung

BS-ÜV8

→ RAID 1 (Mirroring)

⇒ Erhöhung der Ausfallsicherheit

⇒ Festplatten-Verbund von mindestens zwei Festplatten

⇒ Mirroring (Spiegelung)

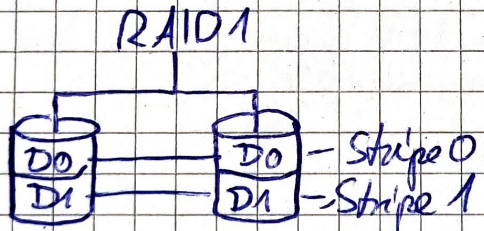
↳ Festplatten werden in Stripes, Daten in Blöcke gleicher Größe aufgeteilt

↳ Blöcke werden auf jeder Festplatte im gleichen Stripe abgelegt

⇒ Zugriffe:

↳ Schreibzugriffe müssen auf allen Festplatten erfolgen, diese können parallel durchgeführt werden

↳ Lesezugriffe können auf jeder Festplatte durchgeführt werden, d.h. zwei unabhängige Lesezugriffe parallel



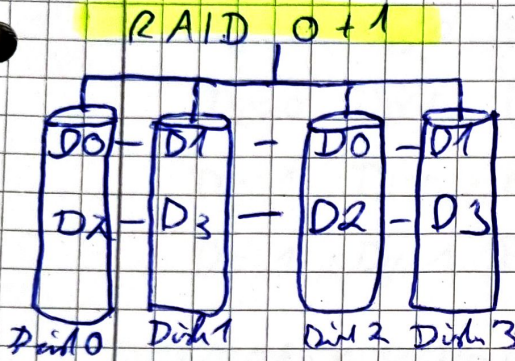
⇒ Verfall: (+) Ausfall einer Festplatte führt zu keinem Datenverlust

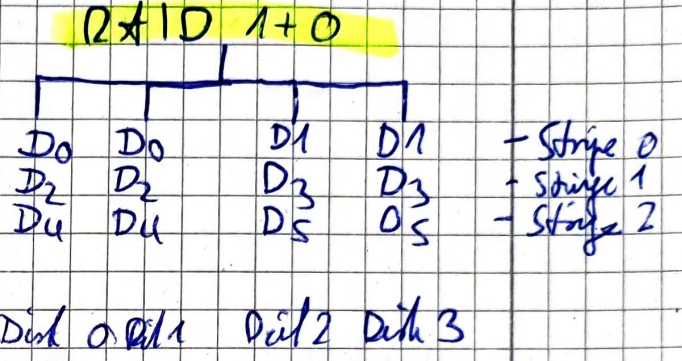
⇒ Nachteile: (-) Kapazität beträgt nur 50% der Festplattenkapazität

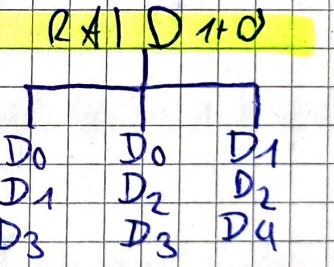
(-) Obwohl Schreibzugriffe parallel ausgeführt werden, dauert gegenüber einer einzelnen Festplatte im Mittel ein Schreibzugriff länger

Varianten von RAID 1

- Kombination aus Striping und Mirroring
- **RAID 0+1 (RAID 01)**
  Zwei Platten bilden ein RAID 0 System welches gespiegelt wird
- **RAID 1+0 (RAID 10)**
  Auf unterster Ebene ist ein RAID 1 System, mehrere Platten sind zu einem RAID 0 System zusammengefasst







RAID Consistent-Update Problem

- Logischer Schreibvorgang muss atomar sein, d.h. Schreibvorgang wird entweder auf allen Platten oder keiner durchgeführt
- RAID Controller arbeitet sequentiell: Disk 1 → Disk 2 → ...
- **Problem:** Stromausfall nach Ende des ersten Schreibvorgangs aber vor Anfang des zweiten
- Lösung: write-ahead log
  - Schreiboperationen werden zunächst in batteriegestütztem RAM-Speicher geschrieben, erst danach werden Schreibaufträge an Platten geschickt
- Nach Stromausfall kann mittels write-ahead log Konsistenz wieder hergestellt werden

BS - MVS

~> RAID 4

-> Redundante Speicherung bei RAID 1 zu platzaufwändig

-> Bei RAID 4 besteht Redundanz aus Paritätsinformationen

-> Für jeden Stripe wird zusätzlicher Block mit Paritätsinformation auf separater Festplatte gespeichert.

-> Parität wird blockweise mit Funktion XOR berechnet

RAID 4

$D_A = 0$	$D_B = 0$	$P_{AB} = 0$
$D_C = 0$	$D_D = 1$	$P_{CD} = 1$
$D_E = 1$	$D_F = 0$	$P_{EF} = 1$
$D_F = 1$	$D_H = 1$	$P_{FH} = 0$

-> Vorteile: (+) RAID 4 kann den Ausfall genau einer Festplatte kompensieren

(+) Nettokapazität: $\frac{n-1}{n}$ % der Gesamtkapazität

(+) Lesezugriffe können auf den n-1 Festplatten parallel durchgeführt werden

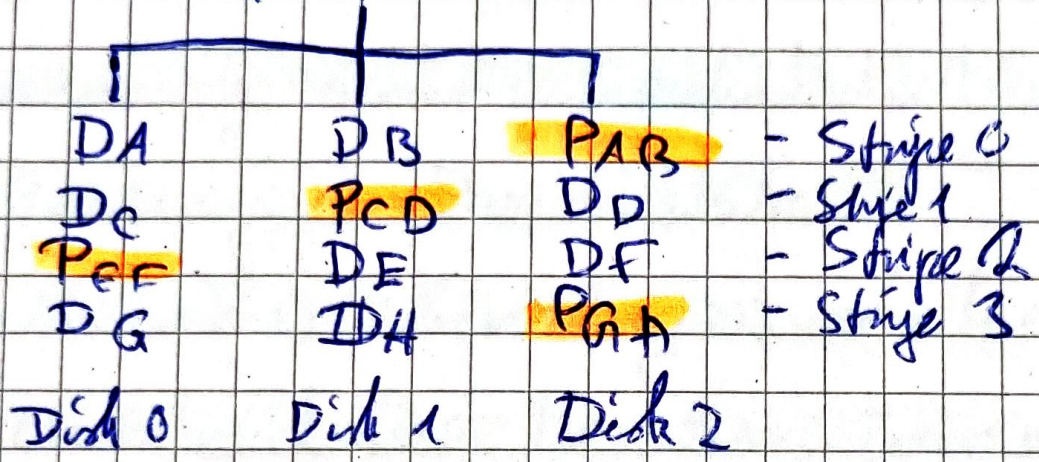
-> Nachteile: (-) Jeder Schreibzugriff muss auf Festplatte mit Paritätsinformationen schreiben (nur sequentiell möglich)

BS-UVB

⇝ RAID 5

↳ Der Nachteil von RAID 4 wird dadurch behoben, dass die Paritätsinformation auf alle Platten verteilt wird, dadurch können auch Schreibzugriffe parallel durchgeführt werden.

RAID 5



BS-KVS

→ RAID 6

- → Verkraftet gleichzeitigen Ausfall von bis zu zwei Festplatten
- → Striping mit doppelten, auf Block-level verteilten Paritätsinformationen
- → Zwei unterschiedliche mathematische Berechnungen für Paritätsblöcke
  - → Verwendet zweier unterschiedlicher Verfahren ~~zum~~ zur Bestimmung der Paritätsblöcke
  - → Neben XOR Berechnung innerhalb eines Stripes noch zusätzlich XOR Berechnung über diagonale Blockreihen. Dabei muss jeder Datenblock durch zwei unterschiedliche Paritätsblöcke abgesichert sein, es dürfen nie 2 Datenblöcke gemeinsam in 2 Paritätsblöcken eingetragen sein
- → Vorteil: ⊕ Verkraftet Ausfall von zwei Festplatten
- → Performance-Nachteil bei Schreiboperationen bei RAID 6 größer als bei RAID 5
- → Netto-Kapazität: $n-2/n$ % der Gesamtkapazität
- → Technisch komplexer

RAID 6

D_A	D_B	D_C	P_{ABC}	Q_{ABC}	– Stripe 0
D_D	D_E	P_{DEF}	Q_{DEF}	D_F	– Stripe 1
D_G	P_{GHI}	Q_{GHI}	D_A	D_i	– Stripe 2

BS – Ü 18

→ Hot-Spare Festplatte:

Im System in Reserve gehaltene Festplatte, wird im Normalbetrieb nicht verwendet

→ Hot-Swap Festplatte:

Die Festplatten eines RAID Systems sind frei zugänglich und können während des laufenden Betriebs ersetzt werden

BS-II/B-GS

S-1

a) Welches ist die entscheidende Leistungsanforderung in einem interaktiven BS?

↪ Kurze Antwortzeiten bei interaktiven Aufträgen

b) Was ist der Unterschied zwischen präemptivem Scheduling und nicht-präemptivem Scheduling?

Beim präemptiven-Scheduling kann einem Prozess die CPU jederzeit entzogen werden.
Beim nicht-präemptiven-Scheduling hat ein Prozess solange die CPU, bis es endet oder freiwillig die CPU abgibt.

↪ Nicht-präemptives Scheduling ist für Systeme mit interaktiven Benutzern nicht geeignet.

↪ Präemptives Scheduling verursacht größeren Aufwand, erlaubt aber Bearbeitung einer höheren Anzahl von Prozessen.

BS-KIB-Ex5

5-1)

c) Was passiert bei einem Prozesswechsel?

1) Prozess anhalten
2) Wechsel in Kernel-Modus
3) PCB aktualisieren
4) Nächsten Prozess auswählen
5) MMU für nächsten Prozess vorbereiten
6) PCB des nächsten Prozesses aktualisieren und dessen Prozesszustand laden
7) Wechsel in User-Modus
8) Ausführung der nächsten Anweisung

d) Thread wechsel?

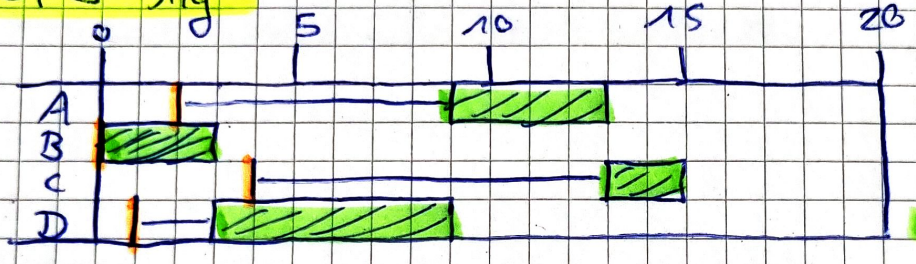
ohne Schritt 5!

BS-Übung 6 x S

S-2) Scheduling

Prozess	Ankunftszeit	Ausführungszeit
A	2	4
B	0	3
C	4	2
D	1	6

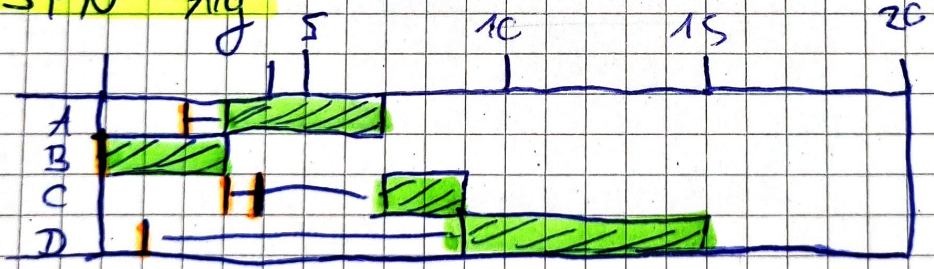
a) FCFS-Alg



→ Benachteiligung kurzer Prozesse

→ kein Verdrängen

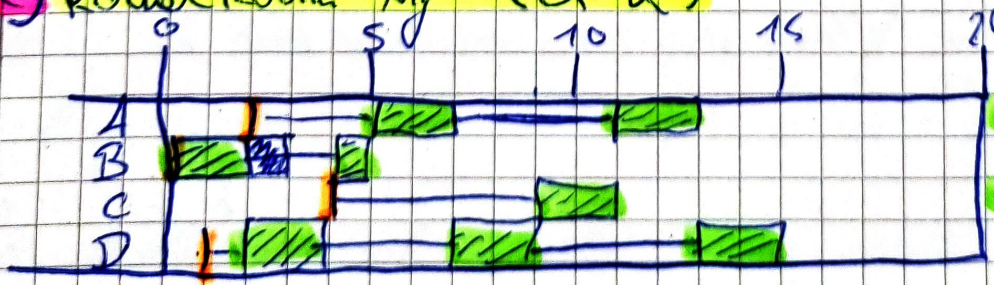
b) SPN-Alg



→ Benachteiligung längere Prozesse

→ Verdrängen möglich

c) Round Robin-Alg (Q=2)



→ Fair

→ kein Verdrängen

~~B~~ ~~D~~ ~~B~~ ~~D~~ ~~B~~ ~~A~~

B A D

A D C

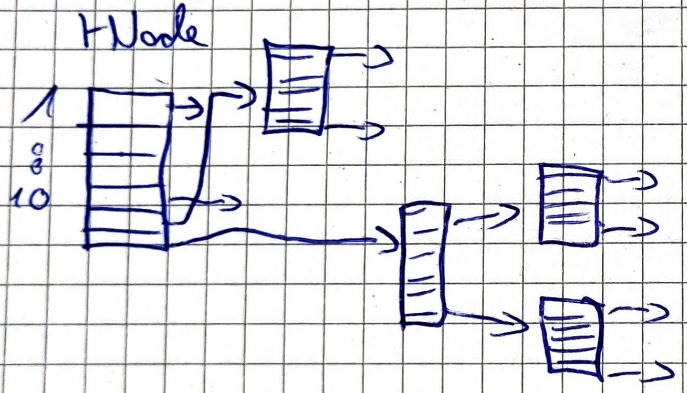
C A

(5-3)

1-Node Dateisystem!
↳ 10 direkte Verweise
↳ einen einfach & zweifach indirekten Verweis.
↳ 2 GiB Festplattenpartition
↳ Blöcke der Größe 4 KiB
↳ Ein Verweise auf einen Block benötigt 8 B

} je in ein Block

a) Maximale Dateigröße dieses Systems.



$\frac{4\,KiB}{8\,B} = 2^9 = 512$ Verweise pro Block.

$(10 + 512 + 512^2) \cdot 4\,KiB = 1\,GiB + 2\,MiB + 40\,KiB$

b) Wie viele Blöcke werden vom Dateisystem benötigt, um die Verwaltungsinformationen für eine Datei mit maximaler Dateigröße zu verwalten?

$1 + 1 + 1 + 512 = 515$ Blöcke
↳ 2056 KiB

BS-KUB-Ex 5

S-3

c) Wie viele Dateien mit der Größe von 6 KiB können in diesem Dateisystem angelegt werden?

Datei mit 6 KiB → braucht zwei Blöcke je 4 KiB

↓

8 KiB

↳ $\frac{2\ \text{GiB}}{8\ \text{KiB}} = \frac{2^{31}}{2^{13}} = 2^{18}$ Dateien

$= 262144$ Dateien

BS - HÜB - Ex S

(5-9) Bewertung von Festplatten

Interessant = Sequentieller & zufälliger Zugriff

↳ Höhe der Anzahl von Blöcken, welche hintereinander auf der Festplatte liegen gelesen wird

↳ Zufällig auf Festplatte verteilt

Ø Seek Time: 7 ms

Ø Drehverzögerung: 3 ms

Datentransferrate: 50 MiB/s

↳ Bestimme Ø Trefferrate

a) S (Sequentiell) Trefferrate: $S = \frac{\text{Menge der Daten}}{\text{Zugriffszeiten}} = 47{,}62 \text{ MiB/s}$

Zugriffszeit: $7\text{ms} + 3\text{ms} + \frac{10 \text{ MiB}}{50 \text{ MiB/s}} = 210\text{ ms}$

b) R (random) Trefferrate: $R = \frac{\text{Menge der Daten}}{\text{Zugriffszeiten}} = 0{,}958 \text{ MiB/s}$

Zugriffszeit: $7\text{ms} + 3\text{ms} + \frac{10 \text{ MiB}}{50 \text{ MiB/s}} = 10{,}195\text{ ms}$

c) $IOPS = \frac{1 \text{ Operation}}{\text{sekt. Latency}} = 100 \text{ Operationen}$

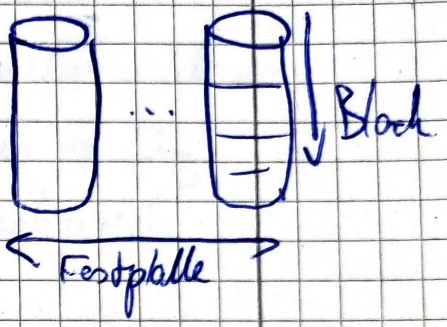
BS-HÜB-Gr 5

(5-5')

1) Betrachten Sei ein RAID 0 System mit n Festplatten und Blockgröße 4 KiB. Geben Sie Formeln an, wie aus einer logischen Blockadresse A die zugehörige Festplatte und die physikalische Blockadresse berechnet wird.

A mod n = zugehörige Festplatte

A / n = Blockadresse



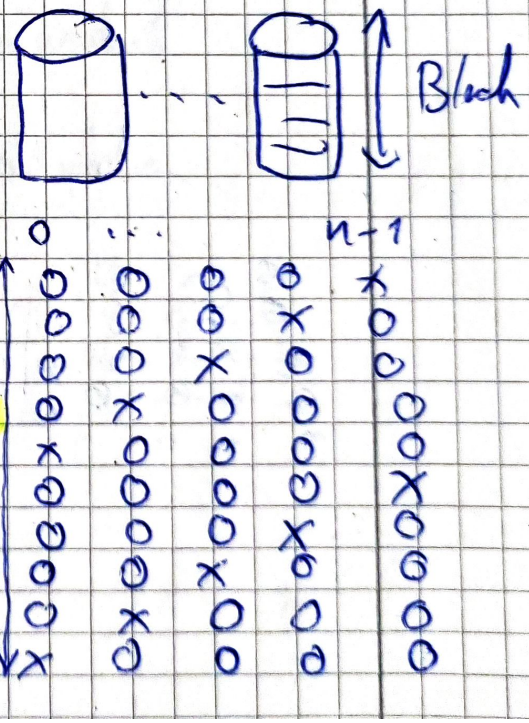
2) Bei RAID 5

Blockadresse: A/(n-1)

Paritätsplatte: $n-1-(\frac{A}{n-1} \% n)$

Plattennummer = A % (n-1)

if (Plattennummer ≥ Paritätsplatte) Plattennummer++



BS - RUD - Ex 5

(S-6) Update bei RAID 4/5

RAID 4/5 mit n Festplatten ein Datenblock D_B überschrieben, so muss Paritätsblock aktualisiert werden

1) Additive Paritäts-Methode:

Hierbei wird zuerst die Stripenummer des zu überschreibenen Blocks D_B ermittelt. Dann werden alle Datenblöcke dieses Stripes bis auf D_B gelesen und aus diesen und D_B wird der neue Paritätsblock P berechnet. Danach werden die beiden Blöcke D_B & P geschrieben.

Wie viele Blöcke müssen gelesen / geschrieben werden?

n-2 Blöcke gelesen & 2 Blöcke geschrieben

2) Subtraktive Paritäts Methode.

$$(a \text{ XOR } b) \text{ XOR } a = b$$

Wie viele Blöcke gelesen / geschrieben?

Neuer Wert: D_B'

$P' = D_B' \text{ XOR } (P \text{ XOR } D_B)$

Lesen = 2 Blöcke, schreiben = 2 Blöcke